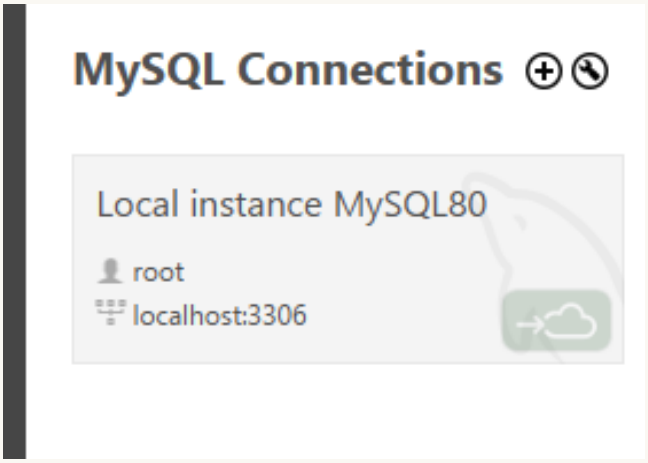
# Conectar Django con MySQL

GUÍA DE PRÁCTICA · CONFIGURACIÓN LOCAL

## Paso 1 · Crear una conexión en MySQL Workbench

La base de datos y el usuario ya fueron creados en MySQL. Abrimos Workbench y, en **MySQL Connections**, presionamos el botón **+** para registrar una nueva conexión.



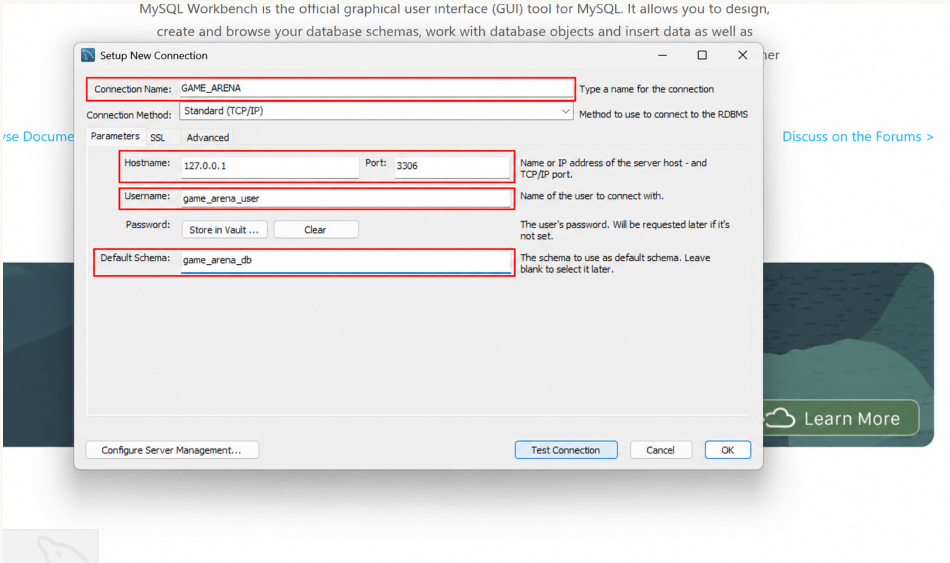

La conexión de Workbench permite comprobar que el servidor acepta las credenciales antes de configurar el proyecto Django.

# Configurar la conexión

## Paso 2 · Completar los datos del servidor

Utilizamos **Standard (TCP/IP)** y completamos los campos con los datos creados previamente.

| Campo | Valor de ejemplo |
|---|---|
| Connection Name | GAME_ARENA |
| Hostname / Port | 127.0.0.1 / 3306 |
| Username | game_arena_user |
| Default Schema | game_arena_db |




Presionamos **Test Connection**, escribimos la contraseña del usuario y confirmamos con **OK**. Si MySQL está en esta computadora usamos 127.0.0.1; para un servidor externo se utiliza su dirección IP o nombre de host.



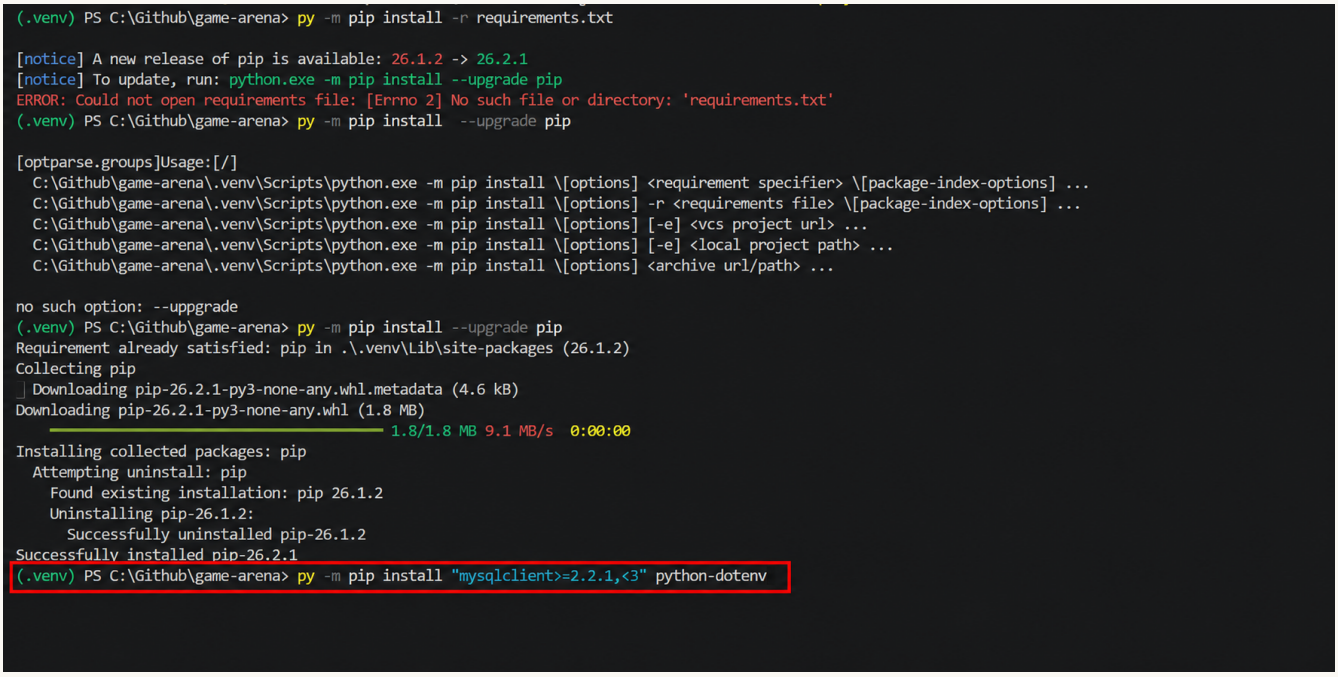
# Instalar el controlador de MySQL

## Paso 3 · Preparar las dependencias de Python

MySQL ya está instalado. Ahora instalamos dentro del entorno virtual el adaptador que permite a Django comunicarse con el servidor y la librería para leer variables de entorno.

```
py -m pip install "mysqlclient>=2.2.1,<3" python-dotenv
```

**mysqlclient** proporciona el controlador de MySQL para Django. **python-dotenv** permite cargar valores desde un archivo **.env**.



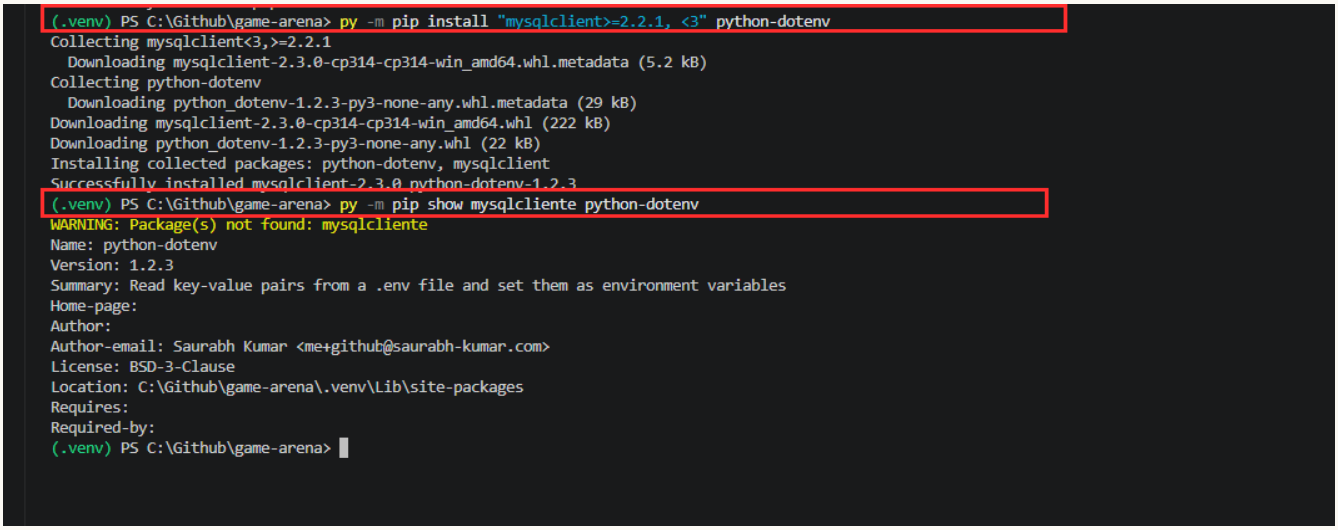

El error de requirements.txt indica que ese archivo no estaba en la carpeta. La opción correcta para actualizar pip es **--upgrade**; actualizar pip es opcional.

# Comprobar la instalación

## Paso 4 · Verificar los paquetes y el controlador

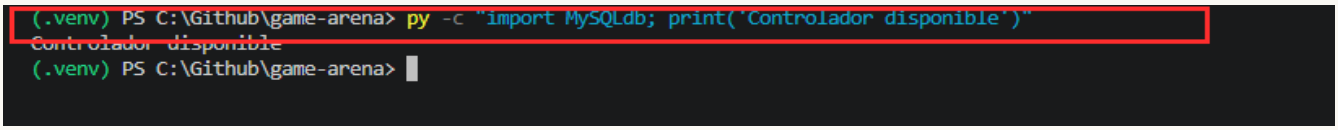
La terminal confirma que se instalaron mysqlclient y python-dotenv. Para consultar ambos paquetes se deben escribir sus nombres exactos:

```
py -m pip show mysqlclient python-dotenv
```




En la captura se escribió **mysqlcliente**; por eso aparece Package(s) not found. También podemos comprobar directamente el módulo proporcionado por mysqlclient:

```
py -c "import MySQLdb; print('Controlador disponible')"
```



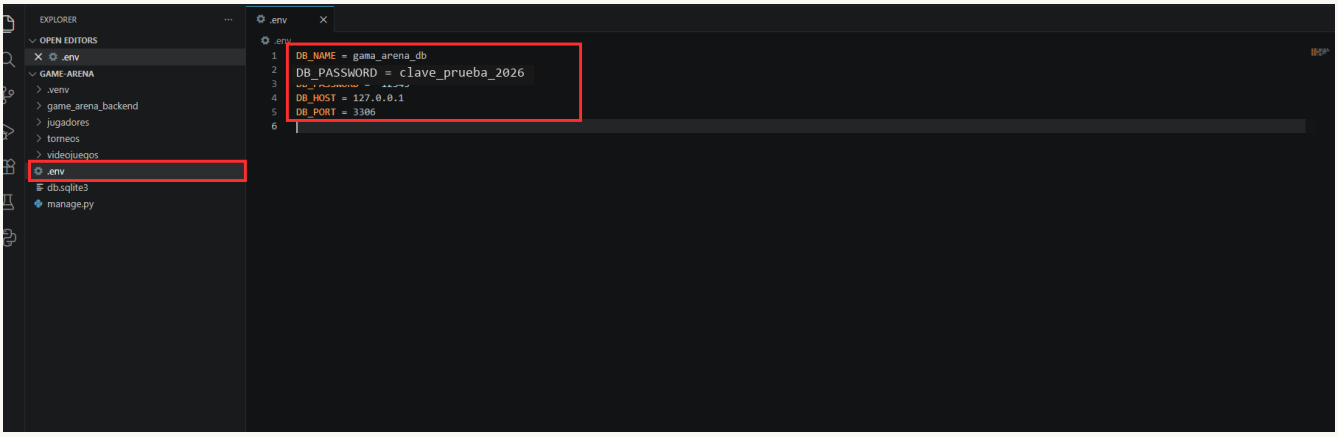

El mensaje **Controlador disponible** confirma que Python puede importar el controlador sin errores.

# Crear el archivo .env

## Paso 5 · Guardar la configuración en variables de entorno

En el explorador de VS Code hacemos clic en el espacio gris debajo de los archivos, creamos un archivo llamado **.env** y lo dejamos en la raíz del proyecto: al mismo nivel que **manage.py** y **db.sqlite3**.

```
DB_NAME=game_arena_db
DB_USER=game_arena_user
DB_PASSWORD=clave_prueba_2026
DB_HOST=127.0.0.1
DB_PORT=3306
```




Estas variables guardan el nombre de la base de datos, el usuario, la contraseña, el host y el puerto. Como el servidor es local usamos **127.0.0.1**, equivalente a localhost. Para un servidor externo se coloca su IP o dominio.

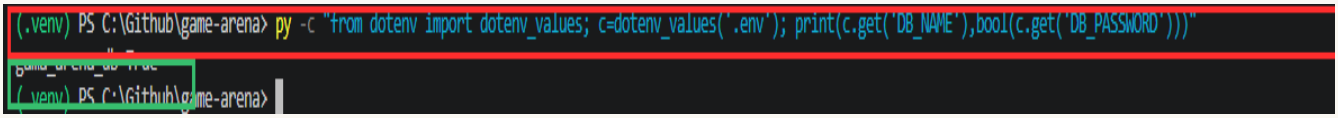
**Importante:** la contraseña mostrada es ficticia. Agregamos **.env** al archivo **.gitignore** para evitar publicar credenciales reales en el repositorio.

# Verificar la lectura del archivo .env

## Paso 6 · Comprobar las variables de entorno

Antes de configurar Django, podemos comprobar que **python-dotenv** encuentra el archivo y lee sus valores. Ejecutamos el siguiente comando desde la carpeta donde se encuentran **.env** y **manage.py**:

```
py -c "from dotenv import dotenv_values; c=dotenv_values('.env');
print(c.get('DB_NAME'), bool(c.get('DB_PASSWORD')))"
```




La salida **game_arena_db True** confirma dos cosas: se leyó correctamente **DB_NAME** y existe un valor para **DB_PASSWORD**.

Utilizamos **bool(...)** para verificar la contraseña sin imprimirla en la terminal. Así evitamos que una credencial real quede visible en capturas, grabaciones o en el historial de comandos.

Si aparece **None** o **False**, revisamos que el archivo se llame exactamente **.env**, que esté en la raíz del proyecto y que los nombres de las variables coincidan con el código.

# Cargar el .env en settings.py

## Paso 7 · Preparar la configuración de Django

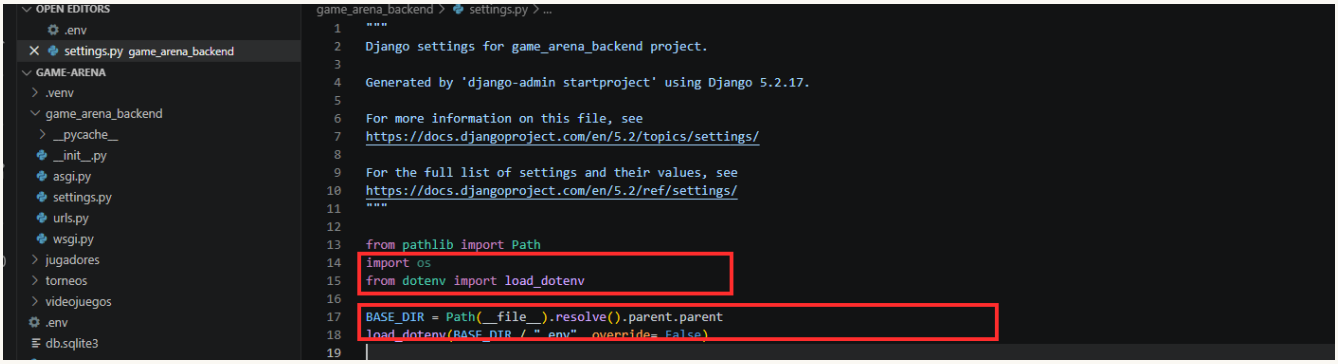
Abrimos **game_arena_backend/settings.py**. Además de importar **os**, importamos la función que leerá el archivo de variables de entorno:

```
import os
from dotenv import load_dotenv
```

Después de definir **BASE_DIR**, cargamos el archivo ubicado en la raíz del proyecto:

```
load_dotenv(BASE_DIR / ".env", override=False)
```

**override=False** conserva una variable que ya exista en el sistema y utiliza el valor del archivo **.env** solamente cuando sea necesario.




Este código debe ejecutarse antes de consultar **os.environ** u **os.getenv** en la configuración de la base de datos.

# Configurar MySQL en DATABASES

## Paso 7 · Sustituir la configuración de SQLite

Buscamos el bloque **DATABASES** y cambiamos el motor por **django.db.backends.mysql**. Los datos sensibles se obtienen desde las variables cargadas del archivo **.env**.

```
DATABASES = {
'default': {
'ENGINE': 'django.db.backends.mysql',
'NAME': os.environ['DB_NAME'],
'USER': os.environ['DB_USER'],
'PASSWORD': os.environ['DB_PASSWORD'],
'HOST': os.getenv('DB_HOST', '127.0.0.1'),
'PORT': os.getenv('DB_PORT', '3306'),
'OPTIONS': {'charset': 'utf8mb4'},
}
}
```




**os.environ** exige que la variable exista y muestra un error si falta. **os.getenv** permite indicar un valor predeterminado; por eso se utiliza para el host y el puerto locales.

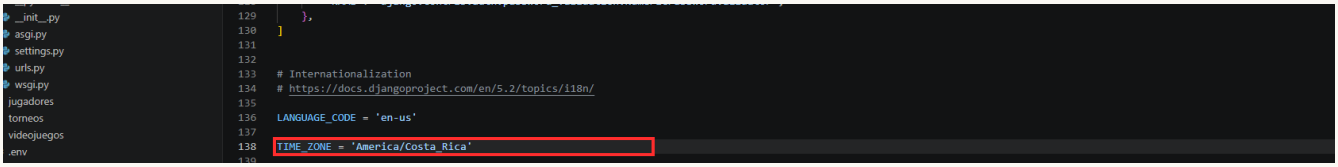
El bloque **TEST** de la captura es opcional y solo se necesita si queremos definir características concretas para la base de datos utilizada durante las pruebas automatizadas.

# Configurar la zona horaria

## Paso 8 · Utilizar el horario de Costa Rica

En el mismo archivo **settings.py** buscamos la sección **Internationalization** y cambiamos el valor de **TIME_ZONE**:

```
TIME_ZONE = 'America/Costa_Rica'
```




**America/Costa_Rica** es el identificador oficial de la zona horaria utilizado por Django y por la base de datos de zonas horarias de Python.

Esta configuración permite que las fechas y horas mostradas por la aplicación se adapten al horario costarricense. Conviene mantener también **USE_TZ = True**, para que Django almacene los valores de forma consistente y realice la conversión al mostrarlos.

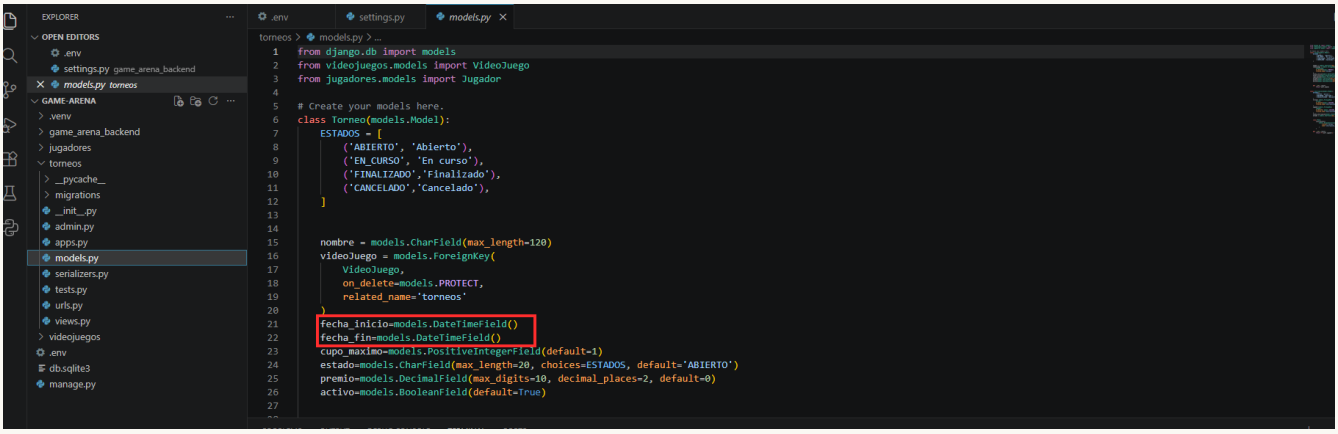
El nombre debe escribirse exactamente como aparece en el código, respetando la barra inclinada y el guion bajo.

# Revisar los campos de fecha

## Paso 9 · Aplicar la zona horaria cuando la aplicación maneja fechas

Este paso solo es necesario cuando la aplicación utiliza fechas u horas. En nuestro caso abrimos **torneos/models.py** y comprobamos que el modelo **Torneo** posee dos campos de tipo **DateTimeField**:

```
fecha_inicio = models.DateTimeField()
fecha_fin = models.DateTimeField()
```




Estos campos almacenan la fecha y la hora de inicio y finalización del torneo. No es necesario cambiar su definición para utilizar el horario de Costa Rica.

Con **TIME_ZONE = 'America/Costa_Rica'** y **USE_TZ = True**, Django conserva las fechas de forma consistente y las convierte a la zona horaria configurada cuando se presentan al usuario.

El formato visual, por ejemplo **día/mes/año hora:minutos**, se define después en el serializer, en un formulario o en la plantilla. El modelo indica el tipo de dato, pero no controla por sí solo cómo se muestra.

Si una aplicación no maneja fechas ni horas, no hace falta agregar campos DateTimeField únicamente por esta configuración.

# Abrir el shell de Django

## Paso 10 · Ingresar a la consola interactiva

Con el entorno virtual activo, abrimos la terminal en la raíz del proyecto, donde se encuentra **manage.py**, y ejecutamos:

```
py manage.py shell
```




Este comando inicia una consola de Python con la configuración y las aplicaciones de Django ya cargadas. Así podemos consultar modelos, comprobar fechas y probar la conexión con la base de datos desde el contexto real del proyecto.

El indicador **>>>** confirma que el shell está listo para recibir instrucciones. El mensaje sobre los objetos importados automáticamente es informativo y no representa un error.

Para salir de la consola podemos escribir **exit()** y presionar Enter. En Windows también es posible usar **Ctrl+Z** y luego Enter.

# Comprobar la conexión con MySQL

## Paso 11 · Ejecutar una consulta desde el shell

Dentro del shell importamos la conexión que Django construyó a partir del bloque **DATABASES** de settings.py:

```
from django.db import connection
```

Luego abrimos un cursor y ejecutamos una consulta SQL pequeña para identificar la base de datos activa y la versión del servidor:

```
with connection.cursor() as cursor:
cursor.execute("SELECT DATABASE(), VERSION()")
print(cursor.fetchone())
```




**connection.cursor()** abre un canal temporal para enviar instrucciones a MySQL. El bloque **with** se encarga de cerrarlo correctamente cuando termina la operación.

**cursor.execute(...)** envía la consulta. **DATABASE()** devuelve la base actualmente seleccionada y **VERSION()** devuelve la versión de MySQL. Finalmente, **fetchone()** recupera la primera fila del resultado.

La respuesta **('game_arena_db', '8.0.46')** confirma que Django está conectado a la base esperada mediante MySQL 8.0.46. El número exacto puede variar según la versión instalada.

Para cerrar correctamente el shell escribimos:

```
exit()
```

Al salir regresamos a PowerShell y podemos continuar ejecutando comandos de Django desde la terminal normal.

# Validar la configuración de la base de datos

### Paso 12 · Ejecutar la comprobación de Django

Después de salir del shell regresamos a PowerShell, mantenemos activo el entorno virtual y ejecutamos:

```
py manage.py check --database default
```




El comando **check** revisa la configuración del proyecto y la conexión definida como **default** en DATABASES.

El mensaje **System check identified no issues (0 silenced)** indica que Django no encontró errores de configuración relacionados con esta base de datos.

# Revisar las migraciones pendientes

### Paso 13 · Consultar el estado y el plan de ejecución

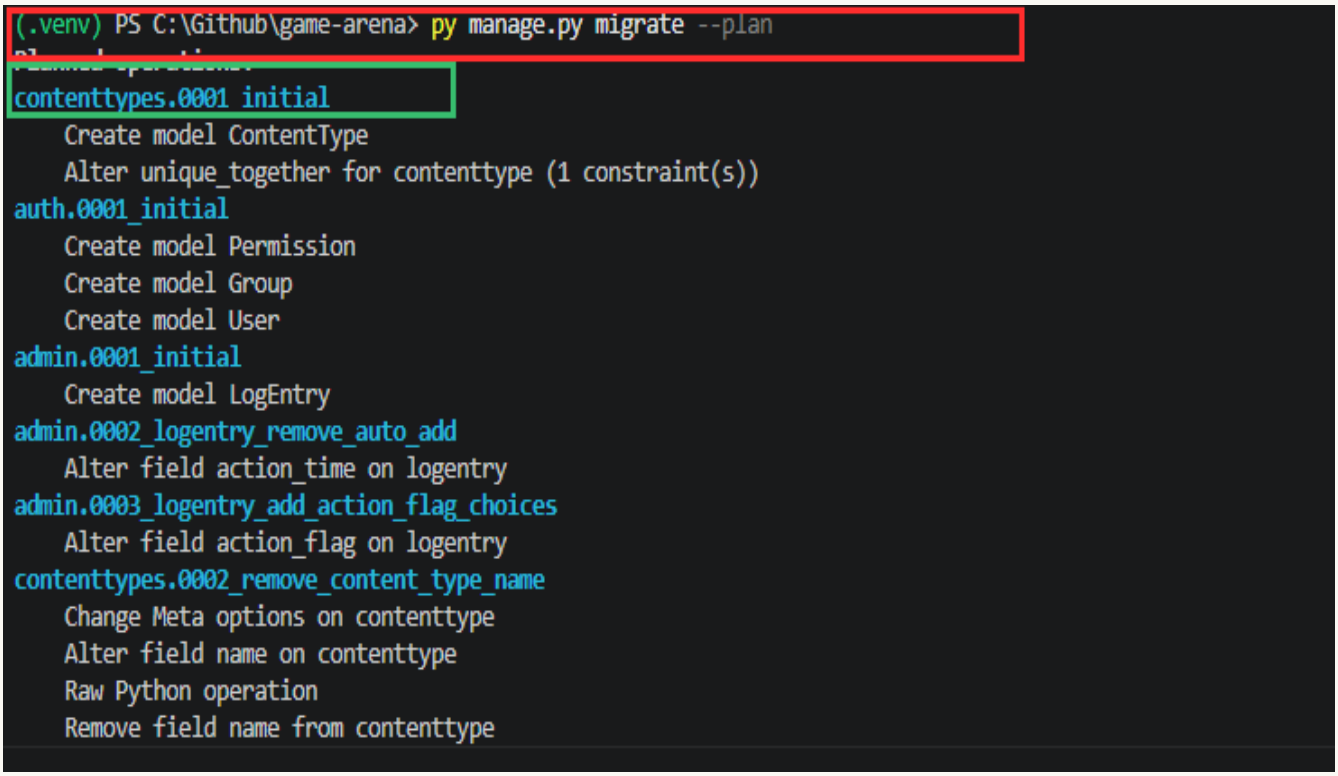
Primero consultamos cuáles migraciones reconoce Django y cuáles ya fueron aplicadas:

```
py manage.py showmigrations
```

La salida muestra cada migración con **[X]** si ya está aplicada y con **[ ]** si todavía está pendiente.

Después revisamos qué operaciones realizaría Django en la base de datos, sin ejecutar cambios todavía:

```
py manage.py migrate --plan
```




La sección **Planned operations** enumera, en orden, las migraciones y las acciones que se aplicarían, como crear modelos, agregar campos o modificar restricciones.

La opción **--plan** es solo informativa. Nos permite revisar el proceso antes de ejecutar la migración real.

# Aplicar las migraciones en MySQL

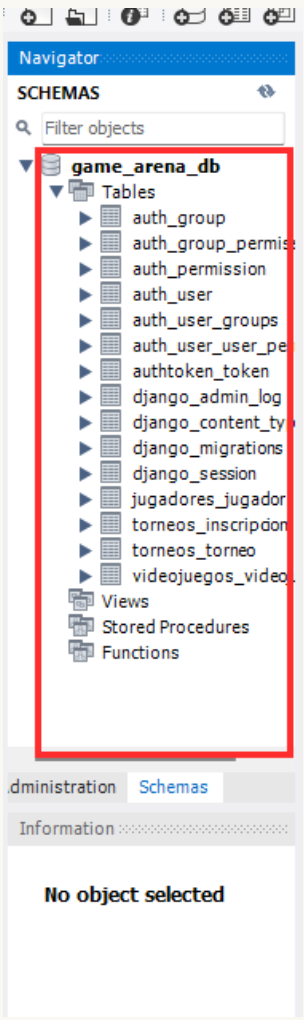
## Paso 14 · Crear las tablas y comprobar el resultado

Después de revisar el plan, ejecutamos las migraciones reales:

```
py manage.py migrate
```

Django toma las migraciones pendientes y crea o actualiza las tablas dentro de la base configurada como **default**. En este caso, los cambios se aplican sobre **game_arena_db** en MySQL.

Al finalizar abrimos MySQL Workbench, entramos en **Schemas** y actualizamos la lista. Expandimos **game_arena_db > Tables** para comprobar el resultado.




La aparición de tablas como **auth_user**, **django_migrations**, **django_session** y las tablas de nuestras aplicaciones confirma que las migraciones se aplicaron correctamente.

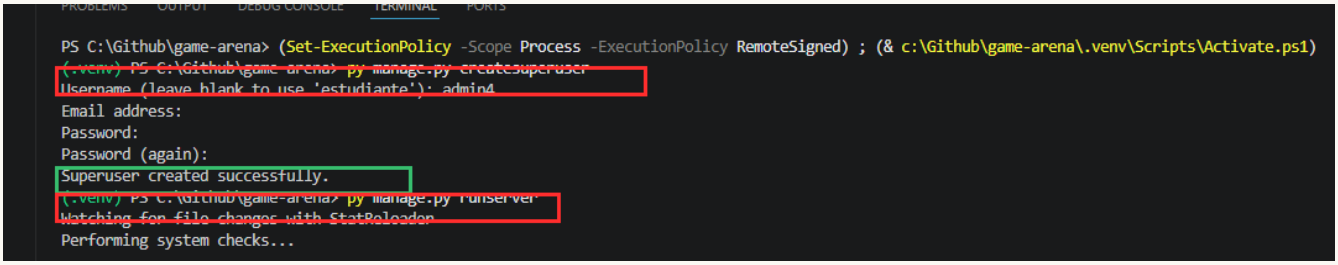
# Crear un superusuario

## Paso 15 · Acceder al panel de administración

Con las tablas creadas, generamos una cuenta administrativa desde la terminal:

```
py manage.py createsuperuser
```

Django solicita el nombre de usuario, el correo electrónico y la contraseña. La contraseña no se muestra mientras la escribimos. Al finalizar debe aparecer **Superuser created successfully**.




Después iniciamos el servidor de desarrollo:

```
py manage.py runserver
```

Abrimos el navegador e ingresamos a:

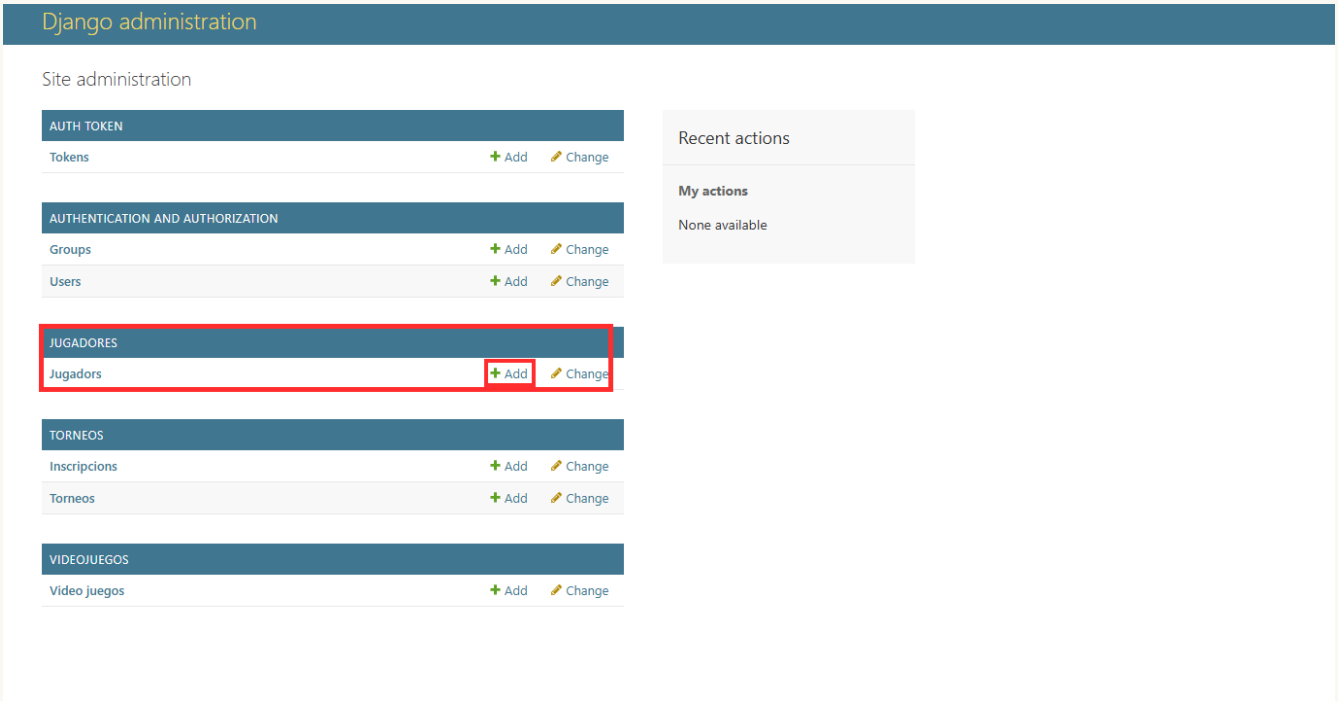
```
http://127.0.0.1:8000/admin/
```

Iniciamos sesión con el superusuario recién creado. Desde el panel podremos administrar los registros guardados en la nueva base de datos MySQL.

# Agregar un jugador desde el panel

## Paso 16 · Comprobar el guardado de información

Iniciamos sesión en el panel de administración con el superusuario. En la página principal buscamos la sección **JUGADORES**.




En la fila **Jugadores** presionamos **Add**. Django abrirá el formulario correspondiente al modelo para completar los datos del nuevo jugador.

Después de llenar los campos obligatorios, presionamos **Save**. Si el registro se guarda correctamente, esto confirma que la aplicación puede insertar información en las tablas creadas dentro de MySQL.

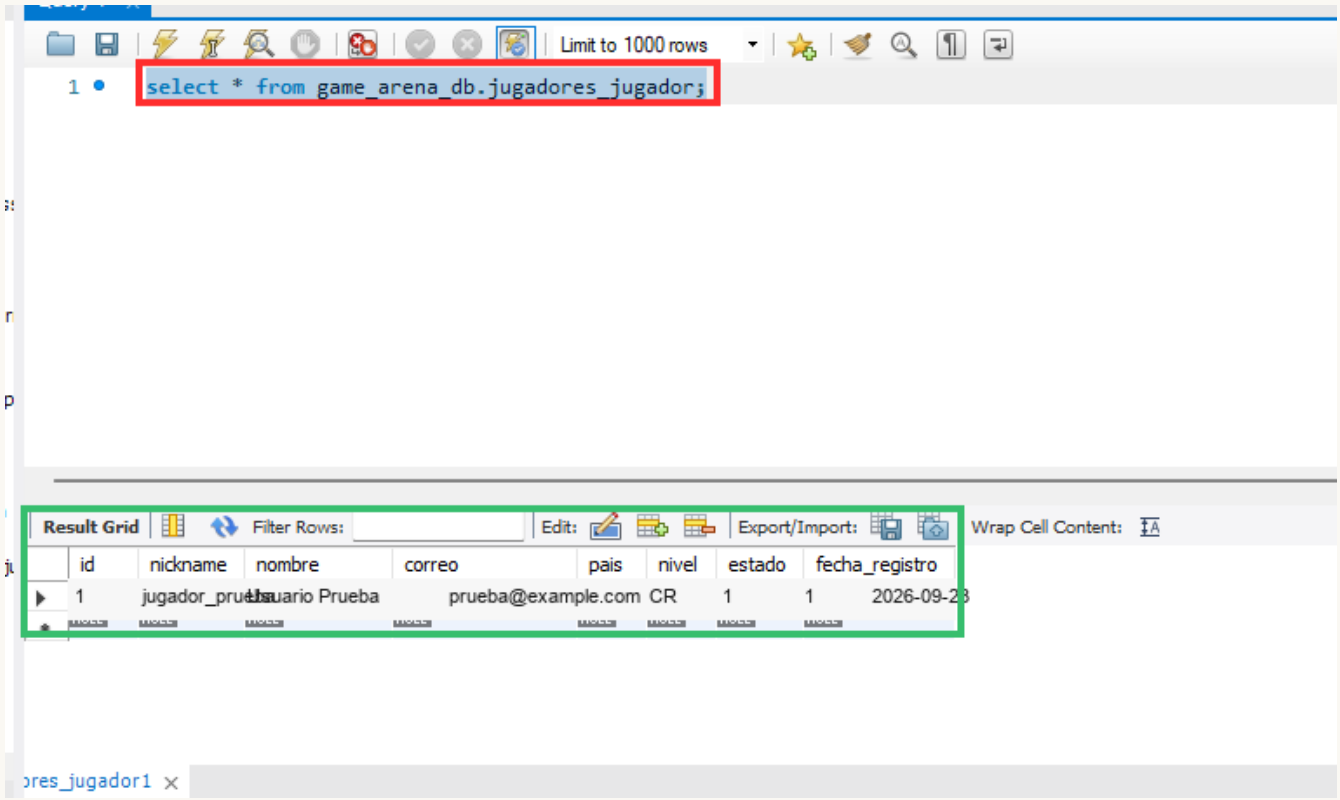
También podemos usar **Change** para consultar, modificar o eliminar jugadores que ya existan.

# Comprobar el registro en MySQL

## Paso 17 · Verificar que el jugador fue almacenado

Regresamos a MySQL Workbench y consultamos directamente la tabla de jugadores:

```
SELECT * FROM game_arena_db.jugadores_jugador;
```




En **Result Grid** aparece el jugador que agregamos desde el panel de Django. La fila muestra sus datos y confirma que el registro quedó almacenado en la tabla **jugadores_jugador** de MySQL.

Esta verificación completa el recorrido: Django recibió la información desde el panel administrativo y la guardó correctamente en el servidor configurado.

Los datos mostrados en la captura de la guía son valores de prueba.